# Ders 15 Çalışma Özeti

## Ders 15 Çalışma Özeti

### Genel Konular

- Vize sınavının biçimi ve ödevlerle ilgili duyurular.
- Öncelikli kuyruk (priority queue): tanım, max ve min öncelikli kuyruk.
- Öncelikli kuyruğun gerçek hayattaki kullanımları: işlem zamanlayıcı (process scheduler), olay güdümlü benzetim (event-driven simulation).
- Heap işlemlerinin tekrarı: `maximum`, `extract_max`, `increase_key`, `decrease_key`, `insert`.
- Max öncelikli kuyruğun C kodu.
- Heap soruları: bir dizi heap mi, $k$'ıncı en küçük eleman, min heap'i max heap'e çevirme.
- Huffman ağacı ile kayıpsız veri sıkıştırma.

### Hocanın Özellikle Vurguladığı Kısımlar

- Öncelikli kuyruk FIFO değildir. Elemanlar bir anahtar (öncelik) değerine göre sıralanır. Banka sıra makinesi ya da acil servisteki triyaj gibi.
- Öncelikli kuyruk heap ile gerçekleştirilir. En öncelikli eleman her zaman köktedir ($O(1)$). Ekleme ve çıkarma $O(\log n)$'dir.
- Yeni eleman eklenirken önce $-\infty$ yazılır. Böylece işlem yarıda kesilse bile heap düzeni bozulmaz. Sonra `increase_key` ile gerçek değere çıkarılır.
- Sınavda bir problemin "düz" çözümü genelde herkesin aklına gelir. Asıl ölçülen, veri yapısının gücünü kullanan daha iyi çözümü görebilmektir.
- Bir problem için en iyi veri yapısını seçmenin hazır kuralı yoktur. Problemin doğasına bakıp hangi işlemin sık yapılacağını düşünmek gerekir. Sonra veri yapısının varyantlarıyla daha da iyileştirilebilir.
- Ödevlerde dosya adlandırma kurallarına uyulmalıdır. Benzerlik kontrol araçları raporları dosya adından üretir.

### Kısa Tekrar Notları

| Max öncelikli kuyruk | Min öncelikli kuyruk |
|---|---|
| Max heap | Min heap |
| `maximum` → `A[1]`, $O(1)$ | `minimum` → `A[1]`, $O(1)$ |
| `extract_max`, $O(\log n)$ | `extract_min`, $O(\log n)$ |
| `increase_key`, $O(\log n)$ | `decrease_key`, $O(\log n)$ |
| `insert`, $O(\log n)$ | `insert`, $O(\log n)$ |
| Örnek: işletim sistemi işlem zamanlayıcı | Örnek: olay güdümlü benzetici (önce olacak olay önce) |

- $k$'ıncı en küçük eleman:
  - Sırala, $k$'ıncı gözü döndür: $O(n^2)$ (selection sort) ya da $O(n \log n)$ (heap sort).
  - Min heap kur, $k$ kez `extract_min`: $O(n + k \log n)$.
  - $k$ elemanlı max heap tut: $O(n \log k)$ zaman, $O(k)$ ek alan.
- Min heap → max heap: son ebeveynden ($\lfloor n/2 \rfloor$) köke kadar her ebeveyne `max_heapify` uygula.
- Huffman: sık karaktere kısa kod, seyrek karaktere uzun kod. Min öncelikli kuyruk ile kurulur.

## Detaylı Açıklamalar

### Duyurular

- Vize bayramdan sonraki pazartesi, ders saatinde (13.00), online.
  - Sınav iki ya da üç ayrı oturumdan oluşur. Oturumlar arasında 5 dakika ara vardır.
  - Örnek düzen: bilgi soruları (yaklaşık 10 soru, 30 dakika), algoritma tasarımı (akış diyagramı ve analiz, 30-40 dakika), belki bir kodlama sorusu.
  - Soru türleri: bilgi, doğru/yanlış, hata düzeltme, kodlama, akış tasarımı, algoritma analizi.
  - En az bir, belki iki problemin çözümü akış diyagramıyla istenecek. Kâğıda çizilip fotoğrafı PDF olarak yüklenebilir.
  - Üç şube ortak sınava girer. Sorular üç şubenin de işlediği konulardan hazırlanır.
- Değerlendirme: tek vize olduğu için vize %35; ödevler ve proje birlikte %25. Hoca, ödevin ağırlığının vizeyi geçemeyeceğini; bunun ölçme-değerlendirme kuralları ve geçmişteki kopya deneyimleri yüzünden olduğunu söyledi.
- 2. ödevin teslimi dersin ertesi günü. Ağacı grafik olarak göstermek için ek kütüphane kullanılabilir. Ama ne amaçla kullanıldığı videoda ve raporda açıkça anlatılmalı. Diziyi yazdırmak da yeterli.

### Öncelikli kuyruk işlemleri

- `maximum`: `return A[1]`. $O(1)$.
- `extract_max`: heap boş değilse kökü kopyala, son elemanı köke taşı, boyutu bir azalt, `max_heapify(A, 1)`. $O(\log n)$.
- `increase_key(A, i, k)`: $k \geq$ mevcut değer olmalı. Değeri yaz, ebeveyninden büyük olduğu sürece yukarı taşı. $O(\log n)$.
  - Örnek: banka sırasında müşteri numarasını girip önceliğini yükselten müşteri.
- `decrease_key` (max heap'te): değeri küçült, `max_heapify` ile aşağı indir.
- `insert`: boyutu artır, son göze $-\infty$ yaz, `increase_key` çağır.
- Hoca, extract işleminin $O(\log n)$'den hızlı olması gereken durumlarda Fibonacci heap gibi başka heap türlerinin kullanılabileceğini söyledi (bkz. ders sonrası notlar).

### Max öncelikli kuyruk C kodu (slaytlardan)

```c
#define INF 100000          /* basitlik için; normalde limits.h INT_MAX */
#define tree_array_size 20
int heap_size = 0;

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int get_right_child(int A[], int index) {
    if (((2 * index) + 1 < tree_array_size) && (index >= 1))
        return (2 * index) + 1;
    return -1;
}

int get_left_child(int A[], int index) {
    if (((2 * index) < tree_array_size) && (index >= 1))
        return 2 * index;
    return -1;
}

int get_parent(int A[], int index) {
    if ((index > 1) && (index < tree_array_size))
        return index / 2;
```

```
    return -1;
}

void max_heapify(int A[], int index) {
    int left_child_index = get_left_child(A, index);
    int right_child_index = get_right_child(A, index);
    int largest = index;

    if ((left_child_index <= heap_size) && (left_child_index > 0))
        if (A[left_child_index] > A[largest])
            largest = left_child_index;

    if ((right_child_index <= heap_size) && (right_child_index > 0))
        if (A[right_child_index] > A[largest])
            largest = right_child_index;

    if (largest != index) {      /* düğüm en büyük değilse heap değildir */
        swap(&A[index], &A[largest]);
        max_heapify(A, largest);
    }
}

void build_max_heap(int A[]) {
    int i;
    for (i = heap_size / 2; i >= 1; i--)
        max_heapify(A, i);
}

int maximum(int A[]) {
    return A[1];
}

int extract_max(int A[]) {
    int maxm = A[1];
    A[1] = A[heap_size];
    heap_size--;
    max_heapify(A, 1);
    return maxm;
}

void increase_key(int A[], int index, int key) {
    A[index] = key;
    while ((index > 1) && (A[get_parent(A, index)] < A[index])) {
        swap(&A[index], &A[get_parent(A, index)]);
        index = get_parent(A, index);
    }
}

void decrease_key(int A[], int index, int key) {
    A[index] = key;
    max_heapify(A, index);
}

void insert(int A[], int key) {
    heap_size++;
    A[heap_size] = -1 * INF;
    increase_key(A, heap_size, key);
}
```

- Çocuk/ebeveyn fonksiyonları, `max_heapify`, `increase_key` ve `decrease_key` slayt karele-

rinden alındı. Sabitler, `swap`, `build_max_heap`, `maximum`, `extract_max` ve `insert` hocanın anlatımına göre yazıldı; slayttaki yazımdan küçük farklar olabilir.
- Çocuk ve ebeveyn fonksiyonları dizinin dışına taşmayı kontrol eder. Taşma varsa $-1$ döner.

Sürücü kodu (`main`) şu adımları yapar:

| Adım | Dizi |
|---|---|
| 20, 15, 8, 10, 5, 7, 6, 2, 9, 1 sırayla `insert` | 20, 15, 8, 10, 5, 7, 6, 2, 9, 1 |
| `increase_key(A, 5, 22)`: 5 → 22, 15 ve 20 ile yer değiştirir | 22, 20, 8, 10, 15, 7, 6, 2, 9, 1 |
| `decrease_key(A, 1, 13)`: 22 → 13, 20 ve 15 ile yer değiştirir | 20, 15, 8, 10, 13, 7, 6, 2, 9, 1 |
| `extract_max` → 20 | 15, 13, 8, 10, 1, 7, 6, 2, 9 |

- Ardından kalan elemanlar sırayla `extract_max` ile büyükten küçüğe yazdırılır.

**Soru 1: Verilen dizi bir heap mi?**

- Heap olmanın iki şartı vardır. İkisi de ayrı fonksiyonla kontrol edilebilir:
  - `is_complete_binary_tree`: Tüm seviyeler dolu, son seviye soldan dolu mu? Dizide boş göz yalnız sonda olmalıdır.
  - `parent_control`: $i = \lfloor n/2 \rfloor$'den 1'e kadar her ebeveyn için `A[i] >= A[2i]` ve (varsa) `A[i] >= A[2i+1]` mi?
- İkisi de doğruysa `return true`. Hoca bunun tek döngüyle de yapılabileceğini, bunu öğrencilere bıraktığını söyledi.

**Soru 2: $k$'ıncı en küçük eleman**

Girdi: $n$ elemanlı, tekrarsız, sırasız bir dizi ve $k \leq n$.

- **Düz çözüm:** Sırala, $k$'ıncı gözü döndür. Selection sort ile $O(n^2)$, heap sort ile $O(n \log n)$.
- **Min heap ile:**
  - Diziden min heap kur.
  - $k$ kez `extract_min` yap. $k$'ıncı çıkan eleman cevaptır.
  - Hoca karmaşıklığı $O((n+k)\log n)$ olarak verdi. $k \ll n$ ise bu $O(n \log n)$ mertebesindedir.
- $k$ **elemanlı max heap ile (daha iyi):**
  - İlk $k$ elemandan bir max heap kur: $O(k \log k)$.
  - Kalan $n-k$ elemanın her birini kökle karşılaştır. Eleman kökten küçükse kökü çıkar, elemanı ekle: her biri $O(\log k)$.
  - Sonunda heap'te dizinin en küçük $k$ elemanı kalır. Bunların en büyüğü, yani $k$'ıncı en küçük eleman, köktedir.
  - Toplam: $O(k \log k + (n-k)\log k) = O(n \log k)$. Ek alan $O(k)$. $k$ küçükse doğrusala çok yakındır.
- Derste bir öğrenci önce kuyruk (queue) kullanmayı önerdi. Hoca kuyruğun yalnız bir uçtan girilip öbür uçtan çıkılan bir yapı olduğunu hatırlattı. Öneri heap ile düzeltilince yukarıdaki max heap çözümüne dönüştü.

**Soru 3: Min heap'i max heap'e çevirme**

- Kökü ya da son elemanı özel olarak seçmeye gerek yoktur.
- Heap oluşturur gibi $\lfloor n/2 \rfloor$'den 1'e kadar tüm ebeveynlere `max_heapify` uygulanır. Yapraklar zaten tek başına heap'tir.
- Max heap'i min heap'e çevirmek için aynı döngüde `min_heapify` kullanılır.

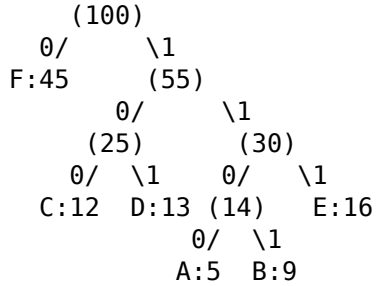
**Huffman ağacı**

- Amaç kayıpsız veri sıkıştırmadır. Sık geçen karakterlere kısa, seyrek geçenlere uzun kod verilir. Zip gibi araçlarda benzeri kullanılır.

- Adımlar:
  1. Metindeki her karakterin frekansını say (sayaç dizisi).
  2. Frekansları anahtar alan bir min öncelikli kuyruk oluştur.
  3. Kuyruktan iki kez `extract_min` yap. İki düğümü çocuk yapan yeni bir ara düğüm oluştur. Frekansı ikisinin toplamıdır, karakteri * gibi özel bir işarettir.
  4. Ara düğümü kuyruğa `insert` et.
  5. Kuyrukta tek düğüm kalana kadar 3-4'ü tekrarla. Kalan düğüm ağacın köküdür.
- Sol kenar `0`, sağ kenar 1 kodunu alır. Kökten karaktere giden yol, karakterin kodudur.

Örnek frekanslar: A = 5, B = 9, C = 12, D = 13, E = 16, F = 45.

| Adım | Çıkarılanlar | Yeni ara düğüm | Kuyruk |
|---|---|---|---|
| 1 | A (5), B (9) | 14 | C12, D13, 14, E16, F45 |
| 2 | C (12), D (13) | 25 | 14, E16, 25, F45 |
| 3 | 14, E (16) | 30 | 25, 30, F45 |
| 4 | 25, 30 | 55 | F45, 55 |
| 5 | F (45), 55 | 100 | 100 |




| Karakter | Frekans | Kod |
|---|---|---|
| F | 45 | `0` |
| C | 12 | `100` |
| D | 13 | `101` |
| E | 16 | `111` |
| A | 5 | `1100` |
| B | 9 | `1101` |

- Örnek metin `FDFEADFC` kodlanır: `0 101 0 111 1100 101 0 100` → toplam 19 bit.
- Kod çözme: bit dizisi soldan okunur, ağaçta yürünür. Bir yaprağa varınca karakter yazılır ve köke dönülür. Hiçbir kod başka bir kodun öneki olmadığı için çözüm tek türlüdür.
- Alıcının kodu çözebilmesi için ağaç da gönderilmelidir. Büyük dosyalarda bu ek yük önemsizdir.

**Ders sonrası notlar**

- **Huffman sıkıştırma oranı:** Hoca 8 karakterlik metni "4 × 8 = 32 bit" ile karşılaştırdı ve yaklaşık %40 kazanç hesapladı. Metinde 8 karakter vardır. 8 bitlik sabit kodla $8 \times 8 = 64$ bit gerekir. 19 bit ile kazanç yaklaşık %70'tir ($45/64$). 6 sembol için 3 bitlik sabit kodla karşılaştırılırsa 24 bit gerekir; kazanç 5 bittir.
- **Fibonacci heap:** Hoca Fibonacci heap'te extract işleminin $O(1)$, ekleme ve güncellemenin $O(\log n)$ olduğunu söyledi. Durum tersidir. Fibonacci heap'te `minimum`, `insert` ve `decrease_key` amortize $O(1)$; `extract_min` amortize $O(\log n)$'dir.
- **Min heap ile $k$'ıncı en küçük:** Heap kurmanın maliyeti $O(n \log n)$ değil, `build_heap` ile $O(n)$'dir. Toplam $O(n + k \log n)$ olur.
- **Hoca max heap çözümünü anlatırken** "minimum 4 elemanı tut" dedi. Burada 4 yalnız örnekteki $k$ değeridir; genel hâli "en küçük $k$ eleman"dır.
- **`extract_max` kodu:** Slayttaki sürüm heap boşken kontrol yapmıyor. Gerçek kodda `heap_size < 1` ise hata döndürülmelidir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.